

Backend Code Structure & Development Pattern Guideline

Purpose

This document defines the standard folder structure and coding pattern that every developer must follow while building backend applications.

The goal is to maintain:

- Clean Architecture
- OOP principles
- Separation of concerns
- Reusable code
- Easy maintenance
- Scalable development
- Better team collaboration

Recommended Project Structure

```
project-root/
|
├── config/
|   ├── database.py
|   ├── settings.py
|   ├── environment.py
|   └── constants.py
|
├── models/
|   ├── user.py
|   ├── role.py
|   ├── product.py
|   └── order.py
|
├── schemas/
|   ├── auth/
|   |   ├── auth.dto.py
|   |   └── token.dto.py
|   |
|   └── user/
|       └── user.dto.py
|
└──
```

```
| | | | | user_response.dto.py
| | | | |
| | | | | product/
| | | | | | | product.dto.py
| | | | |
| | | | | routes/
| | | | | | | auth.routes.py
| | | | | | | user.routes.py
| | | | | | | product.routes.py
| | | | | | | order.routes.py
| | | | |
| | | | | repositories/
| | | | | | | base.repository.py
| | | | | | | user.repository.py
| | | | | | | product.repository.py
| | | | | | | order.repository.py
| | | | |
| | | | | services/
| | | | | | | auth.service.py
| | | | | | | user.service.py
| | | | | | | product.service.py
| | | | | | | order.service.py
| | | | |
| | | | | middleware/
| | | | | | | auth.middleware.py
| | | | | | | permission.middleware.py
| | | | | | | error.middleware.py
| | | | |
| | | | | security/
| | | | | | | jwt.py
| | | | | | | password.py
| | | | | | | encryption.py
| | | | |
| | | | | helpers/
| | | | | | | email.helper.py
| | | | | | | otp.helper.py
| | | | | | | file.helper.py
| | | | | | | response.helper.py
| | | | |
| | | | | exceptions/
| | | | | | | custom.exception.py
| | | | | | | error.handler.py
| | | | |
| | | | | utils/
| | | | | | | logger.py
| | | | | | | validator.py
```

```
|
|— migrations/
|
|— tests/
|
|— main.py
|
|— requirements.txt
|
|— README.md
```

Architecture Flow

Every API request must follow this flow:

```
Client Request
|
|
v
Routes Layer
|
|
v
Middleware
(Authentication,
Authorization)
|
|
v
Service Layer
(Business Logic)
|
|
v
Repository Layer
(Database Operations)
|
|
v
Models
(Database Tables)
|
|
```

v
Database

Folder Responsibilities

1. Config Folder

Purpose:

Contains all application configurations.

Example:

```
config/  
├─ database.py  
├─ settings.py  
└─ constants.py
```

Responsibilities:

- Database connection
 - Environment variables
 - Application settings
 - Global constants
-

2. Models Folder

Purpose:

Contains database models/entities.

Example:

```
models/  
├─ user.py  
├─ product.py  
└─ order.py
```

Rules:

- Only database structure
- No business logic
- No API logic
- No validation logic

Example:

```
User Model
```

```
id
name
email
password
created_at
updated_at
```

3. DTO / Schema Folder

Purpose:

Data Transfer Objects.

Used for:

- Request validation
- Response formatting
- API contracts

Example:

```
schemas/
├──
└── auth/
    ├── auth.dto.py
    └── token.dto.py
```

Rules:

DTO should define:

- Input fields
- Output fields
- Validation rules

DTO should not contain:

- Database queries
 - Business logic
-

4. Routes Folder

Purpose:

Contains only API endpoints.

Example:

```
routes/  
  
auth.routes.py  
user.routes.py  
product.routes.py
```

Responsibilities:

- Define API URL
- Receive request
- Call middleware
- Call service methods
- Return response

Example:

```
POST /login  
  
Route  
|  
AuthService.login()  
|  
Return Response
```

Routes should NOT contain:

- SQL queries
 - Business rules
 - Complex calculations
-

5. Repository Folder

Purpose:

Database communication layer.

Example:

```
repositories/  
  
user.repository.py  
product.repository.py
```

Responsibilities:

- Create records
- Update records
- Delete records
- Fetch records
- Database queries

Example:

```
UserRepository  
  
create_user()  
  
find_by_email()  
  
update_user()  
  
delete_user()
```

Repository only handles:

```
Application
|
Repository
|
Database
```

6. Service Folder

Purpose:

Contains business logic.

Example:

```
services/

auth.service.py
user.service.py
```

Responsibilities:

- Business rules
- Workflow handling
- Calling repositories
- Using helpers
- Using DTOs

Example:

```
AuthService

login()

{
    validate user

    check password

    generate JWT token
```



```
    return response
}
```

Service connects:

```
Route
|
Service
|
Repository
|
Database
```

7. Middleware Folder

Purpose:

Request filtering before reaching services.

Examples:

```
middleware/

auth.middleware.py

permission.middleware.py
```

Responsibilities:

- JWT verification
- User authentication
- Role checking
- Request validation

Example:

```
Request
|
JWT Middleware
```

```
|  
Route  
  
|  
Service
```

8. Security Folder

Purpose:

Security-related functionality.

Example:

```
security/  
  
jwt.py  
password.py  
encryption.py
```

Contains:

- JWT generation
- JWT verification
- Password hashing
- Encryption/decryption

Example:

```
generate_token(user_id)  
  
verify_token(token)
```

9. Helpers Folder

Purpose:

Common reusable functions.

Example:

```
helpers/  
email.helper.py  
otp.helper.py  
file.helper.py
```

Examples:

Email:

```
send_email()
```

OTP:

```
generate_otp()  
verify_otp()
```

File:

```
upload_file()  
delete_file()
```

OOP Development Rules

Class Based Development

Every major module should use classes.

Example:

```
class UserService:  
    def create_user():  
    def update_user():
```

```
def delete_user():
```

Dependency Injection Pattern

Avoid:

```
UserService()
```

inside every file.

Prefer:

```
UserService(  
    UserRepository()  
)
```

Benefits:

- Easy testing
- Loose coupling
- Better maintenance

Naming Convention

Files

Use:

```
user.service.py  
  
user.repository.py  
  
auth.middleware.py
```

Avoid:

```
UserServiceFile.py
```

Classes

Use PascalCase:

```
UserService  
  
AuthRepository  
  
JwtMiddleware
```

Functions

Use camelCase or snake_case depending on language:

Python:

```
create_user()  
get_user_by_id()
```

TypeScript:

```
createUser()  
getUserById()
```

Go:

```
CreateUser()  
GetUserByID()
```

API Development Standard

Example:

Login API

POST /api/auth/login

Flow:

auth.routes

|
v

AuthService.login()

|
v

UserRepository.findByEmail()

|
v

User Model

|
v

Database

|
v

JWT Generate

|
v

Response DTO

Database Rules

Never write SQL queries inside:

- Routes
- Controllers
- Middleware

Allowed:

Repository Layer Only

Example:

```
user.repository.py  
  
get_user_by_email()  
  
create_user()  
  
update_user()
```

Git Development Rules

Each feature should follow:

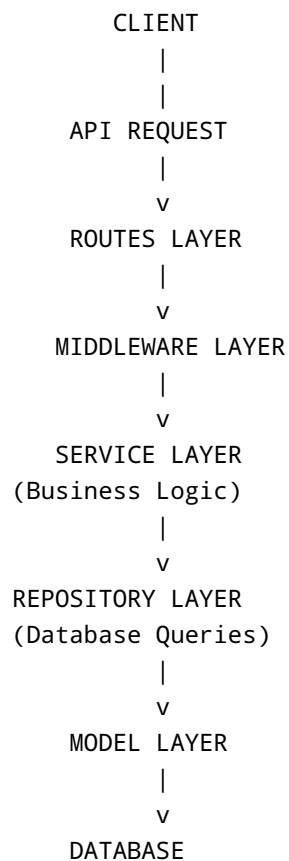
```
feature/  
|  
├─ route  
├─ dto  
├─ service  
├─ repository  
└─ model
```

Example:

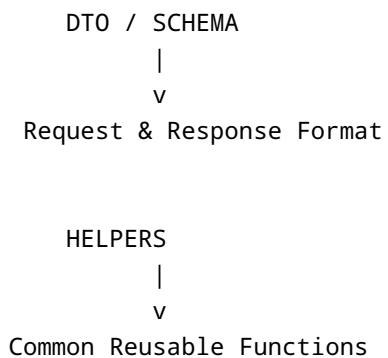
```
feature-user-management  
  
auth  
user
```

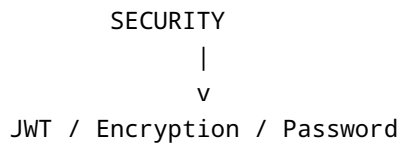
product
payment

Final Architecture Diagram



Supporting Layers:





Team Rule

Every developer must follow this architecture.

No developer should:

- ✗ Write SQL in routes
- ✗ Write business logic in controllers/routes
- ✗ Duplicate helper functions
- ✗ Mix authentication with business logic
- ✗ Create random folders

All new modules must follow:

```
Route
↓
Middleware
↓
DTO
↓
Service
↓
Repository
↓
Model
↓
Database
```

This pattern is mandatory for all backend projects.